FPT ACADEMY INTERNATIONAL

FPT – APTECH COMPUTER EDUCATION

EduCenter Management System

Supervisor: *TRƯƠNG TRUNG*

Semester: *II*

Batch No: *T3.2511.M0*

Group No: *3*

Order:	**Full name**	**Roll No.**
1.	*THÁI PHÁT*	*Student1693582*
2.	*LA VĨ QUYỀN*	*Student1693584*

***Month**: October **Year**: 2026*

This is to certify that

Mr. **THÁI PHÁT**

Mr. **LA VĨ QUYỀN**

Have successfully Designed & Developed

EDUCENTER MANAGEMENT SYSTEM

Submitted by:

Mr. TRUONG TRUNG

Date Of Issue:

Authorized Signature:

Content

Content

Acknowledge

Synopsis

Review 1 ..... Page 01

Review 2 ..... Page 10

Review 3 ..... Page 30

Testing Document ..... Page 38

Final Checklist ..... Page 41

Task Sheet ..... Page 43

Acknowledge

As we understand, the eProject is a step-by-step, guided learning experience that closely mirrors classroom and lab-based training, translating it into actual implementation. In Semester II the eProject allows us to:

- Apply Python, FastAPI and SQL knowledge to a complete, realistic back-end system.
- Design a relational database in PostgreSQL and manage its evolution with migrations (Alembic).
- Package and deploy a multi-container application with Docker and Docker Compose.
- Integrate an applied AI service (Google Gemini) safely, with a rule-based fallback.
- Practise automated testing, role-based security and team work through Git and pull requests.
- Turn individual lab exercises into one unified, working application.

We would like to send a great thank you to our instructor, Mr. Truong Trung, for his guidance and feedback during this project, and to our classmates for their support.

Despite the mistakes we may have made, we hope to keep widening our knowledge of back-end development, DevOps and applied AI.

We also thank Aptech for the project requirements, which helped us gain realistic experience. We will continue learning to be even more successful in the future.

Yours Sincerely, Team Group 03.

Synopsis

The primary objective of this program is to provide hands-on experience in working on real-life projects. These applications serve as building blocks that help us develop larger and more robust systems.

Rather than focusing on one specific tool, the program presents a real-world scenario that requires us to combine the subjects of Semester II: Python programming, database design, containerisation with Docker and applied AI.

The application developed in this project, EduCenter Management System, is a centralised platform for a language and training centre. It replaces a legacy back end built on Firebase Realtime Database and Cloud Functions with a REST API written in FastAPI, a PostgreSQL database and a Docker deployment. The system manages users and permissions for four roles (Admin, Teacher, Student, Parent), classes and enrolments, attendance, scores and class reports, tuition, and an AI Coach that gives feedback on students' writing and speaking texts.

A clear understanding of the subject matter is essential for successful implementation. We have gone through the requirements and designed the system step by step, as described in the reviews of this documentation.

Review 01

Problem Definition

Training centres handle a large amount of daily operational data: classes and timetables, student attendance, test scores, tuition payments and communication with parents. When this data is kept in spreadsheets, chat groups or a loosely structured cloud database, it becomes hard to keep consistent, hard to report on, and hard to protect.

Our reference system stored everything in Firebase Realtime Database, with access rules written in a JSON file and business logic spread across Cloud Functions. This approach created several problems:

- ✓ Data is stored as nested JSON nodes, so joins and statistics (attendance rate, average score, overdue tuition) must be computed by hand in the client or in many functions.
- ✓ Permissions live in a rules file that is difficult to test and easy to break.
- ✓ There is no schema or migration history, so data quality depends on each developer.
- ✓ The system is tied to one vendor and cannot be run offline or on a school server.

Solution: EduCenter Management System rebuilds the back end as a REST API with FastAPI, stores data in a relational PostgreSQL database with Alembic migrations, checks every permission on the server, and runs the whole stack (database, API, web interface) with one Docker Compose command.

1.1 Core Objectives

- ✓ **Centralised data:** store users, classes, enrolments, attendance, scores, tuition and AI Coach history in one relational database.
- ✓ **Secure access:** JWT login, bcrypt password hashing, temporary lock after 5 failed logins, and role-based data scope checked on every request.
- ✓ **Academic operations:** teachers take attendance for a whole class in one step and record scores; students and parents follow progress.
- ✓ **Reporting:** class summaries (attendance rate, average score) computed with pandas and exported to Excel.

- ✓ **Tuition control:** create tuition items, record payments and mark overdue items automatically.
- ✓ **Applied AI:** an AI Coach gives feedback on writing and speaking texts with Google Gemini, with a rule-based fallback and a daily usage limit.
- ✓ **Easy deployment:** the full system runs with docker compose up --build.

1.2 User Roles & Key Functionalities

1. Administrator

- ✓ Manage user accounts for all roles, activate or deactivate accounts.
- ✓ Link parents with their children.
- ✓ Create classes, assign teachers, enrol or remove students.
- ✓ Create tuition items, record payments, mark overdue items, view tuition summary.
- ✓ View dashboards and use the AI Coach.

2. Teacher

- ✓ View only the classes he or she is in charge of.
- ✓ Take attendance for the whole class with four statuses: present, late, absent, excused.
- ✓ Record, edit and delete scores.
- ✓ View the class report and export it to Excel.
- ✓ Use the AI Coach to prepare feedback.

3. Student

- ✓ View own attendance history and attendance rate.
- ✓ View own scores and average per class.
- ✓ View own tuition status.
- ✓ Submit writing or speaking texts to the AI Coach (20 requests per day).

4. Parent

- ✓ Choose one of his or her children.
- ✓ View the child's attendance, scores and tuition. Access to any other student is blocked with HTTP 403.

Project Impact: moving from a loosely structured cloud database to a relational, tested and containerised system makes the centre's data consistent, its permissions verifiable and its deployment repeatable on any machine.

Customer Requirements Specifications

I. CRS/Requirements Specifications

Teacher / Student / Parent Undertaken:

1. List of inputs to the system

- Login credentials (username, password).
- Attendance sheet of a class for a date (student, status, note).
- Score entries (class, student, kind, value, maximum value, note).
- Writing or speaking text for the AI Coach (kind, text).
- Selected child (for parents).

2. Overview of processes involved in the system

- Validate credentials and return a JWT token.
- Save attendance as an upsert (one record per student per day).
- Validate and save scores (value between 0 and the maximum).
- Generate AI feedback (Gemini or rule-based) and save the session.
- Filter every query by the user's data scope.

3. List of outputs expected from the system

- Login result or lock notification.
- Class roster, attendance list and attendance rate.
- Score list and average score per class.
- Class report on screen and as an .xlsx file.
- Tuition status of the student or the selected child.
- AI feedback and history.

Administrator Undertaken:

1. List of inputs to the system

- User data (username, full name, role, password).
- Parent to child links.
- Class data (code, name, schedule, teacher) and enrolments.
- Tuition items (student, class, amount, due date, note) and payments.

2. Overview of processes involved in the system

- Add, edit, activate or deactivate users.

- Create and update classes, enrol and remove students.
- Record payments, mark overdue items, summarise tuition.

3. List of outputs expected from the system

- User list and class list.
- Tuition summary by status and tuition export (.xlsx).
- Overall dashboard of the system.

Hardware and software required for implementing the project

No	Item	Requirement
1	Operating system	Windows 10/11 (64-bit), macOS or Linux
2	Hardware	CPU with 2 cores or better, 8 GB RAM (about 2 GB free for the containers), 5 GB disk
3	Runtime	Docker Desktop (Docker Engine + Docker Compose v2), Git
4	Back end	Python 3.12, FastAPI 0.115, SQLAlchemy 2.0, Pydantic 2, Alembic 1.14, PyJWT, bcrypt, pandas, openpyxl, httpx
5	Database	PostgreSQL 16 (container); SQLite for automated tests
6	Front end	React 19, Vite 7, Tailwind CSS 3, React Router 7, served by nginx
7	AI service	Google Gemini API (gemini-2.5-flash), optional; rule-based fallback without a key
8	Tools	VS Code, Postman / Swagger UI, pytest, GitHub
9	Other	A working internet connection (first build and Gemini calls only)

II. CRS/Customer Acceptance Criteria

S.N	Customer's Acceptance Criteria
1	**Given any user** when he or she logs in with a correct username and password then the system returns a token and opens the dashboard of his or her role.

	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 3	TRUONG TRUNG
Signature		
Date		

S.N	Customer's Acceptance Criteria
2	**Given any user** when the password is wrong 5 times in a row then the account is locked for 15 minutes and login returns HTTP 423.
3	**Given a request without a valid token** when it calls a protected endpoint then the system returns HTTP 401.
4	**Given the Admin role** when managing users then the admin can create, edit, activate, deactivate users and link a parent to a child.
5	**Given the Admin role** when managing classes then the admin can create a class, assign a teacher, enrol and remove students.
6	**Given the Teacher role** when saving attendance twice for the same class and date then the records are updated, not duplicated.
7	**Given the Teacher role** when opening a class he or she does not teach then the system returns HTTP 403.
8	**Given the Teacher role** when entering a score greater than the maximum value then the system rejects it; otherwise the class average is updated.
9	**Given the Teacher role** when exporting the class report then an .xlsx file with attendance rate and average score is downloaded.
10	**Given the Admin role** when running Mark overdue then unpaid items with a past due date change to overdue; paying an item changes it to paid.
11	**Given the Parent role** when viewing data then only his or her own children are visible; any other student returns HTTP 403.

S.N	Customer's Acceptance Criteria
12	**Given the Student role** when submitting a text to the AI Coach then feedback is returned and saved; after 20 requests in a day the system returns HTTP 429.
13	**Given no Gemini API key or a failed Gemini call** when the AI Coach is used then rule-based feedback is returned with provider = rules.

III. Project Plan

1. Project Details

- **Name of the Client:** EduCenter, a language and training centre (EduCenter Management System).
- **Date of Project Plan:** October 2026.
- **Project Vision / Objectives:** Rebuild the centre management back end with FastAPI, PostgreSQL and Docker so that data is relational, permissions are enforced on the server and the system can be deployed with one command.
- **Scope:** Five modules: (1) login and role-based authorisation, (2) classes and enrolments, (3) attendance, scores and reports, (4) tuition, (5) AI Coach. Chat, notifications and gamification of the reference system are out of scope.
- **Our understanding of the client organisation:** The centre runs many classes taught by different teachers. Teachers record attendance and scores after each session, the office manages tuition, and parents want to follow their children's progress. All data used in this project is fictitious.
- **Project Organisation with Responsibilities and Authorities:** Project Team Leader and Developer: La Vĩ Quyền (repository owner; back end, front end, Docker, CI). Analyst, Database Designer and Developer: Thái Phát (requirements, database design, design documentation, remaining screens and tests of modules 1 to 3). Work is split evenly; both members review each other's pull requests and test each other's modules. Details are given in the Task Sheet.

2. Project Initiation / Requirement Documents

- **Documents:** Problem Definition, Customer Requirements Specifications, Project Plan.

3. Deliverables

- **Items:** Design documentation, source code (back end, front end, migrations, tests), Docker Compose configuration, installation guide, user guide, presentation and demo video.

4. Project Dependencies

- **Items:** Docker Desktop on the demo machine; Python 3.12 and Node 20 images; optional Google Gemini API key.

5. Major Project Milestones

- **Project start, requirement analysis:** 09/10/2026
- **Working prototype of 5 modules (back end, front end, Docker):** 14/10/2026
- **Review 1 and Review 2 (1st Status Report):** 23/10/2026
- **Review 3 (screens, testing):** 13/11/2026
- **Submit design documents to the lecturer:** 20/11/2026, 08:00 to 17:30
- **Submit the completed project:** 02/12/2026, 08:00 to 17:30
- **Project defence:** 05/12/2026, 13:30

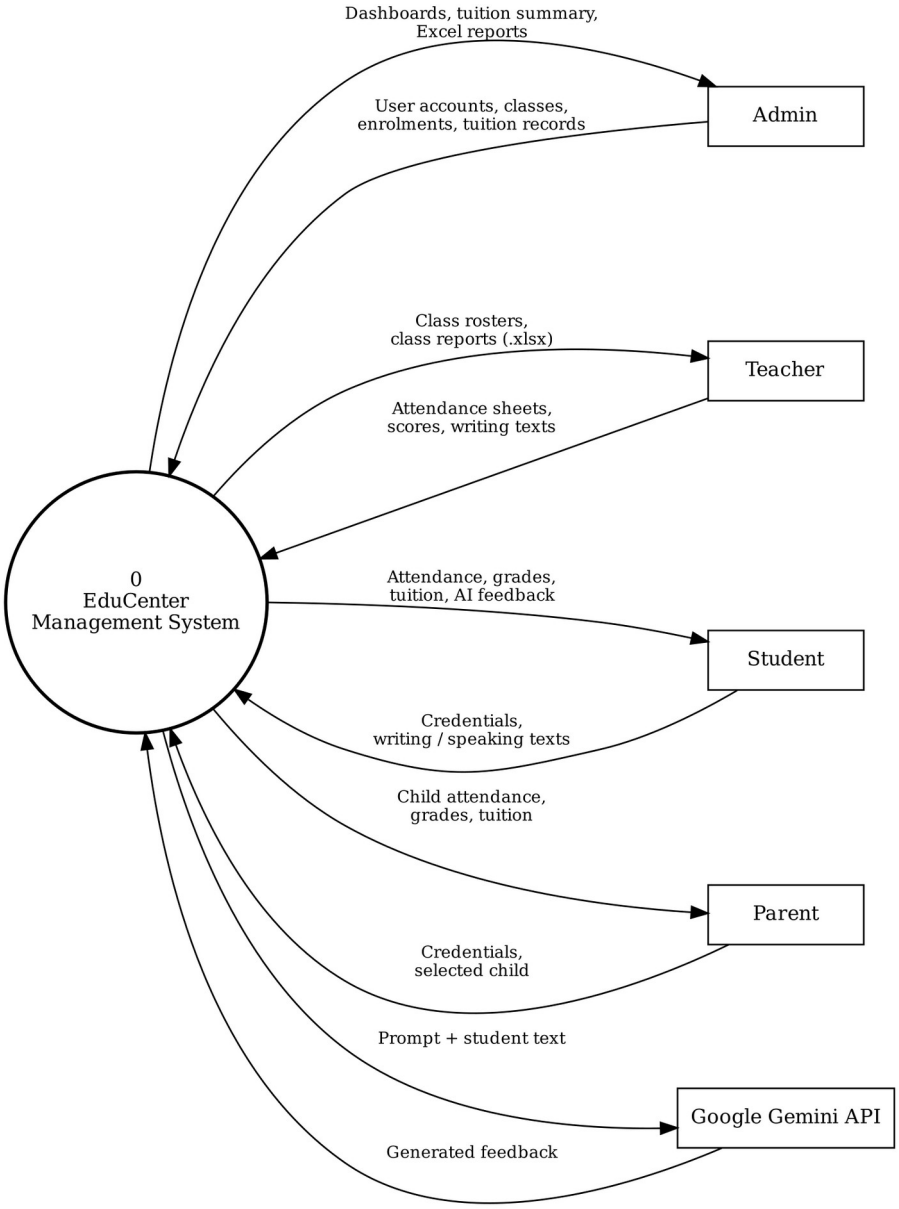
6. Quality Plan

- **Review activities:** Weekly review meeting with the lecturer; every change merged through a pull request reviewed by the other member.
- **Testing activities:** Automated API tests with pytest (15 tests, all passing); manual test cases in the Testing Document; cross testing between members; a clean run from git clone on another machine before submission.
- **Backup and recovery strategies:** Source code on GitHub with feature branches; database stored in a named Docker volume; schema recreated from Alembic migrations and sample data from the seed script (docker compose down -v, then up).

Review 02

Data Flow Diagram

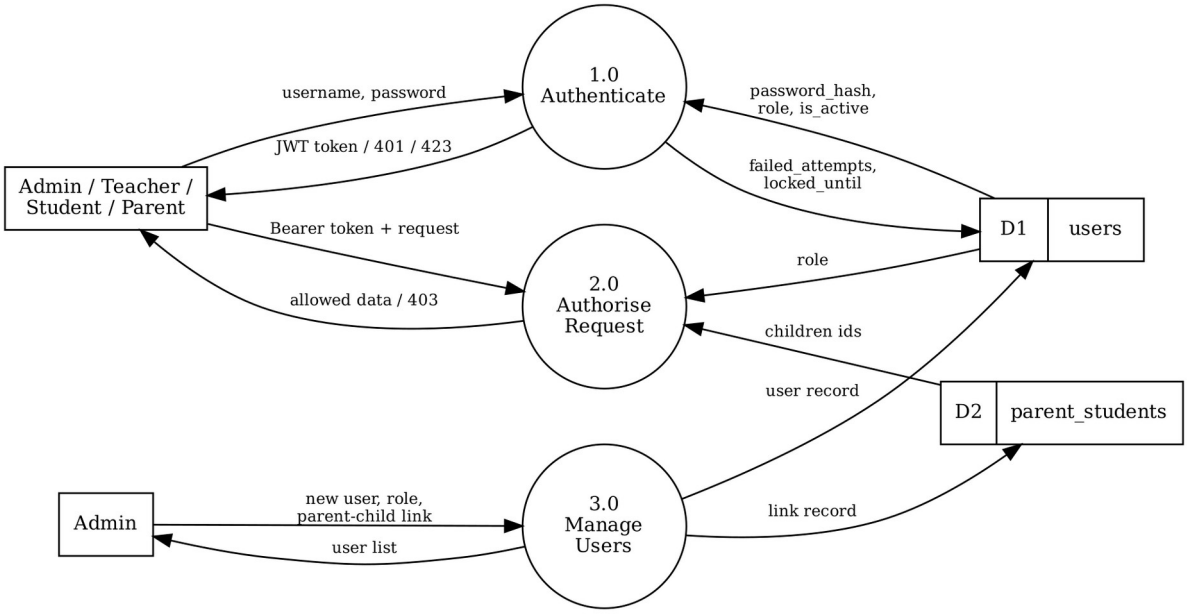
I. Data Flow Diagram Level 0 (Context Diagram)



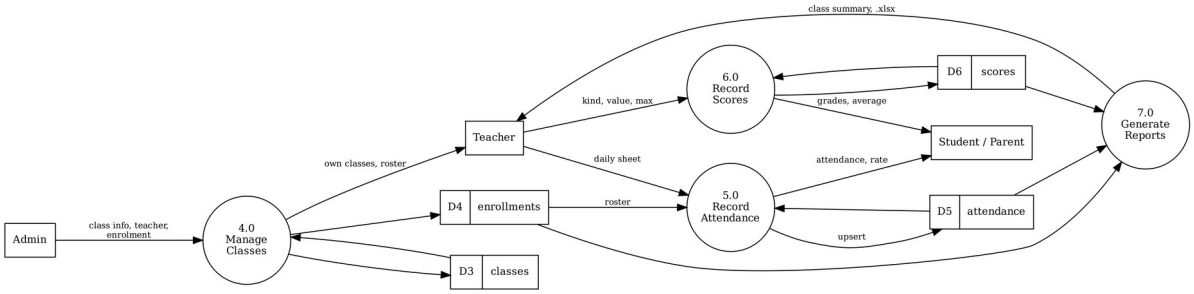
II. Data Flow Diagram Level 1

Level 1 is split into three parts so that every process and data store stays readable.

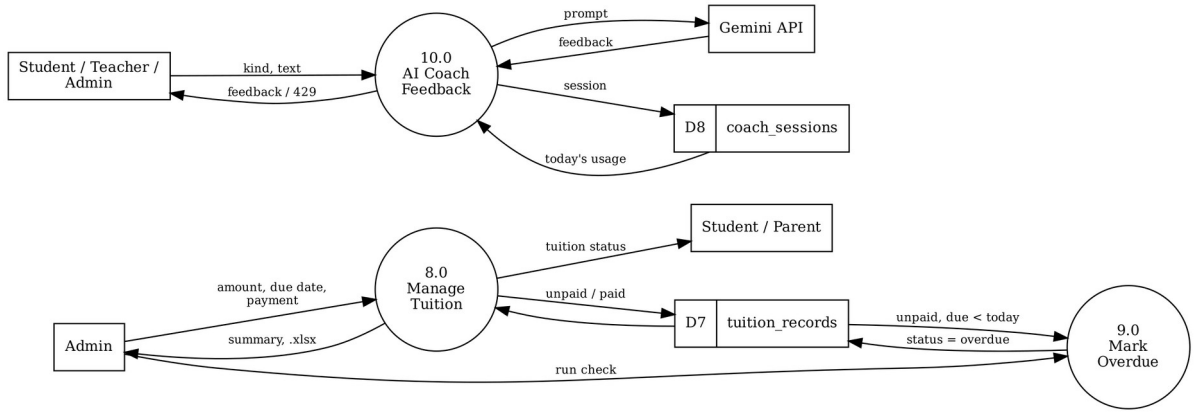
a) Authentication, authorisation and user management



b) Academic operations: classes, attendance, scores and reports



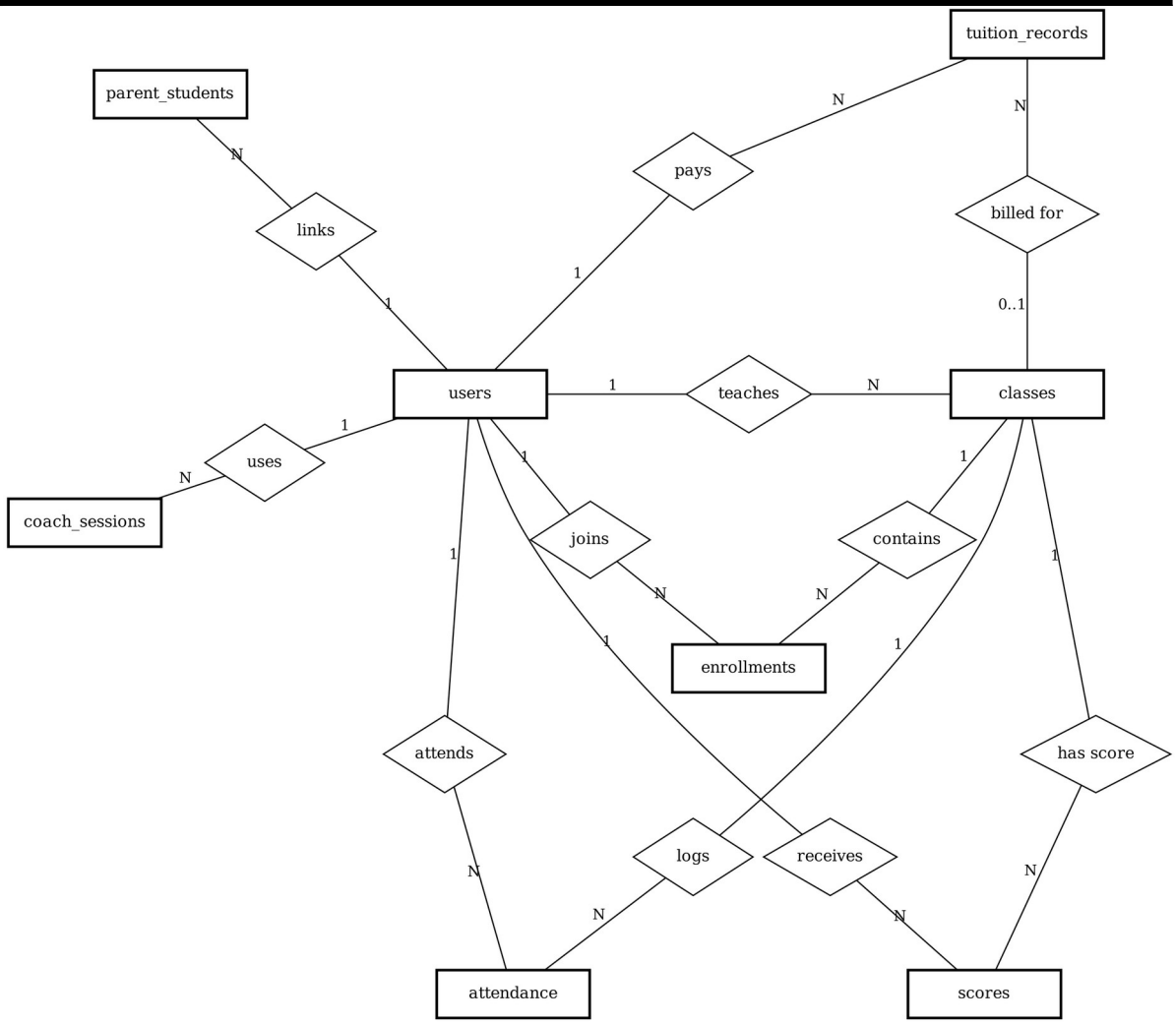
c) Tuition and AI Coach



Data stores

Store	Table	Content
D1	users	Accounts of all roles, password hash, login lock data
D2	parent_students	Parent to child links
D3	classes	Class code, name, schedule, teacher
D4	enrollments	Students enrolled in each class
D5	attendance	One record per student, class and date
D6	scores	Score entries with kind and maximum value
D7	tuition_records	Tuition items and payment status
D8	coach_sessions	AI Coach input, feedback and provider

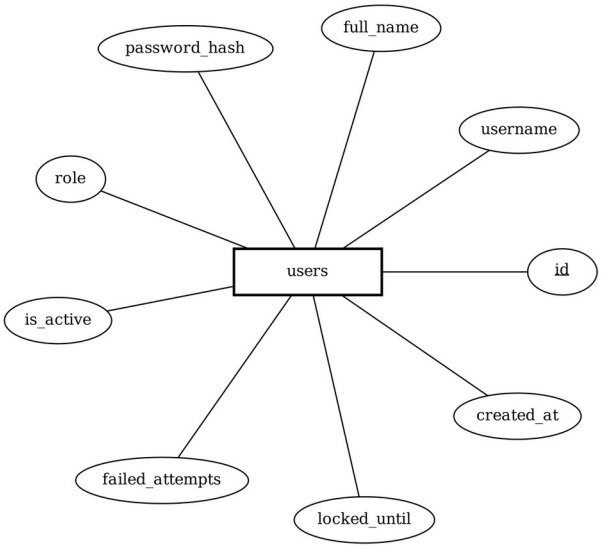
A. Entity Relationship Diagram



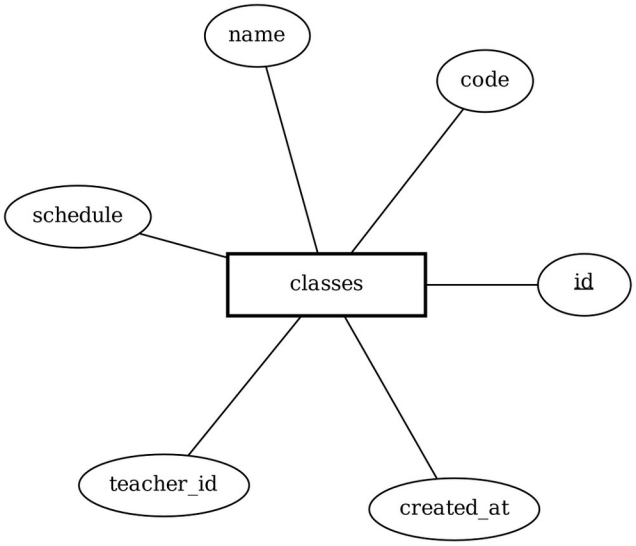
Cardinality: one user (teacher) teaches many classes; one class contains many enrolments, attendance records and scores; one student has many enrolments, attendance records, scores and tuition items; a parent is linked to many children through parent_students; one user has many AI Coach sessions.

B. Entity Attributes

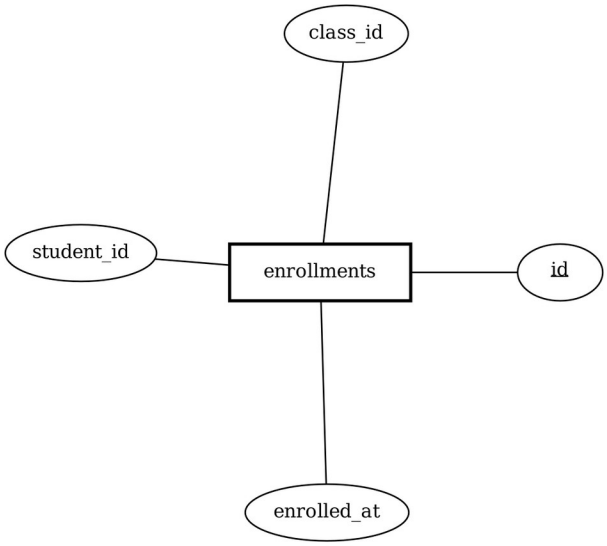
1. User Entity (users)



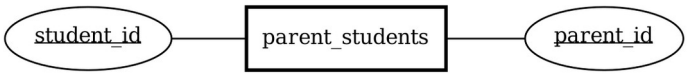
2. Class Entity (classes)



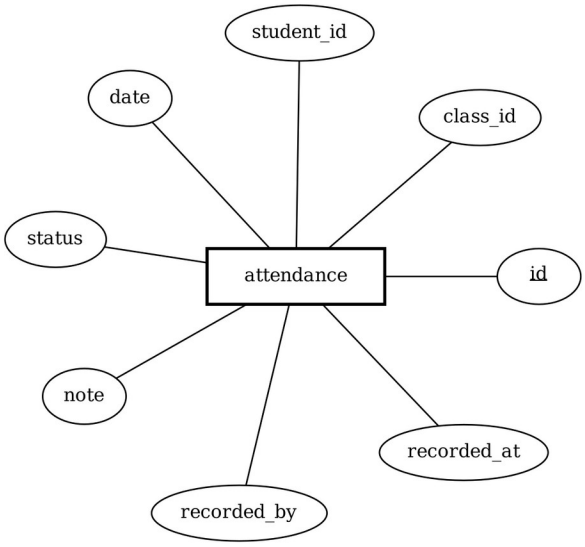
3. Enrolment Entity (enrollments)



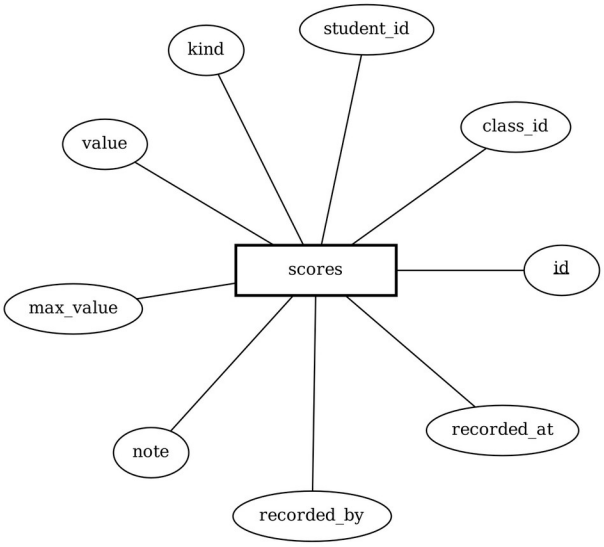
4. Parent - Student Link Entity (parent_students)



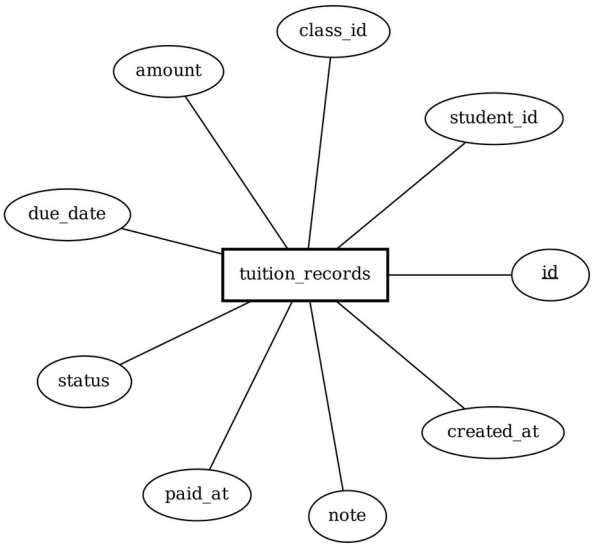
5. Attendance Entity (attendance)



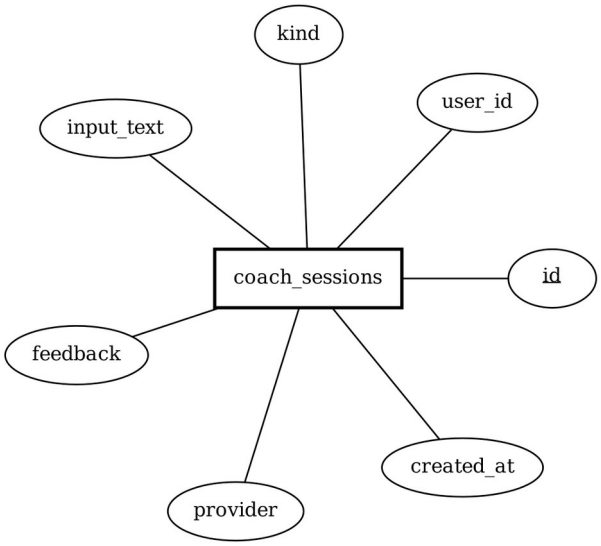
6. Score Entity (scores)



7. Tuition Entity (tuition_records)



8. AI Coach Session Entity (coach_sessions)


kind
user_id
input_text
coach_sessions
id
feedback
created_at
provider

A. Table Design

Table Name: users

Serial	Fields	Data type	Keys	Constraints	Description
1	id	integer	PK	Auto increment, Not Null	User identifier
2	username	varchar(64)		Unique, Index, Not Null	Login name (letters, digits, _ . -)
3	full_name	varchar(120)		Not Null	User's full name
4	password_hash	varchar(255)		Not Null	bcrypt hash of the password
5	role	varchar(16)		Not Null, in (admin, teacher, student, parent)	User role
6	is_active	boolean		Not Null, Default true	Account is active
7	failed_attempts	integer		Not Null, Default 0	Consecutive failed logins
8	locked_until	timestamptz		Nullable	End of temporary lock
9	created_at	timestamptz		Not Null, Default now (UTC)	Record creation time

Table Name: parent_students

Serial	Fields	Data type	Keys	Constraints	Description
1	parent_id	integer	PK, FK	References users.id, On delete cascade	Parent account
2	student_id	integer	PK, FK	References users.id, On delete cascade	Child (student) account

Table Name: classes

Serial	Fields	Data type	Keys	Constraints	Description
1	id	integer	PK	Auto increment, Not Null	Class identifier
2	code	varchar(32)		Unique, Index, Not Null	Class code, e.g. IELTS-A1
3	name	varchar(120)		Not Null	Class name
4	schedule	varchar(200)		Nullable	Timetable description
5	teacher_id	integer	FK	References users.id, On delete set null, Index	Teacher in charge
6	created_at	timestamptz		Not Null, Default now (UTC)	Record creation time

Table Name: enrollments

Serial	Fields	Data type	Keys	Constraints	Description
1	id	integer	PK	Auto increment, Not Null	Enrolment identifier
2	class_id	integer	FK	References classes.id, On delete cascade, Index	Class
3	student_id	integer	FK	References users.id, On delete cascade, Index	Student
4	enrolled_at	timestamptz		Not Null, Default now (UTC)	Enrolment time
5	(class_id, student_id)		UQ	Unique uq_enrollment	A student is enrolled once per class

Table Name: attendance

Serial	Fields	Data type	Keys	Constraints	Description
1	id	integer	PK	Auto increment, Not Null	Attendance identifier
2	class_id	integer	FK	References classes.id, On delete cascade, Index	Class
3	student_id	integer	FK	References users.id, On delete cascade, Index	Student
4	date	date		Not Null, Index	Session date
5	status	varchar(16)		Not Null, in (present, absent, excused, late)	Attendance status
6	note	varchar(300)		Nullable	Teacher's note
7	recorded_by	integer	FK	References users.id, On delete set null	User who recorded
8	recorded_at	timestamptz		Not Null, Default now (UTC)	Recording time
9	(class_id, student_id, date)		UQ	Unique uq_attendance_day	One record per student per day

Table Name: scores

Serial	Fields	Data type	Keys	Constraints	Description
1	id	integer	PK	Auto increment, Not Null	Score identifier
2	class_id	integer	FK	References classes.id, On delete cascade, Index	Class
3	student_id	integer	FK	References users.id, On delete cascade, Index	Student
4	kind	varchar(40)		Not Null	Score type: quiz, midterm, final...

Serial	Fields	Data type	Keys	Constraints	Description
5	value	numeric(5,2)		Not Null, >= 0 and <= max_value	Score obtained
6	max_value	numeric(5,2)		Not Null, Default 10, > 0	Maximum score
7	note	varchar(300)		Nullable	Teacher's note
8	recorded_by	integer	FK	References users.id, On delete set null	User who recorded
9	recorded_at	timestamptz		Not Null, Default now (UTC)	Recording time

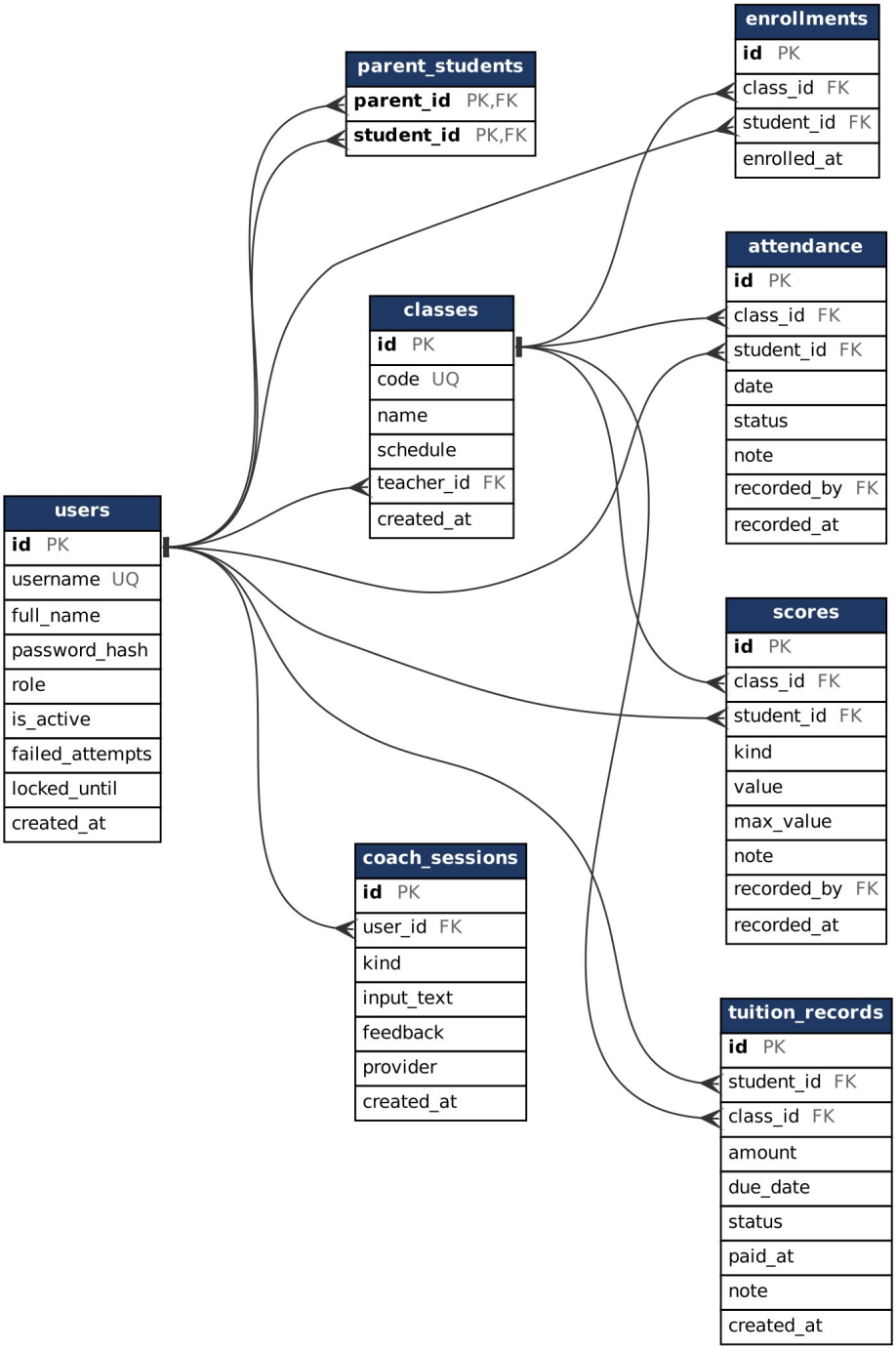
Table Name: tuition_records

Serial	Fields	Data type	Keys	Constraints	Description
1	id	integer	PK	Auto increment, Not Null	Tuition item identifier
2	student_id	integer	FK	References users.id, On delete cascade, Index	Student
3	class_id	integer	FK	References classes.id, On delete set null, Nullable	Related class
4	amount	numeric(12,2)		Not Null, > 0	Amount to pay (VND)
5	due_date	date		Not Null, Index	Payment deadline
6	status	varchar(16)		Not Null, Default unpaid, in (unpaid, paid, overdue)	Payment status
7	paid_at	timestamptz		Nullable	Payment time
8	note	varchar(300)		Nullable	Note
9	created_at	timestamptz		Not Null, Default now (UTC)	Record creation time

Table Name: coach_sessions

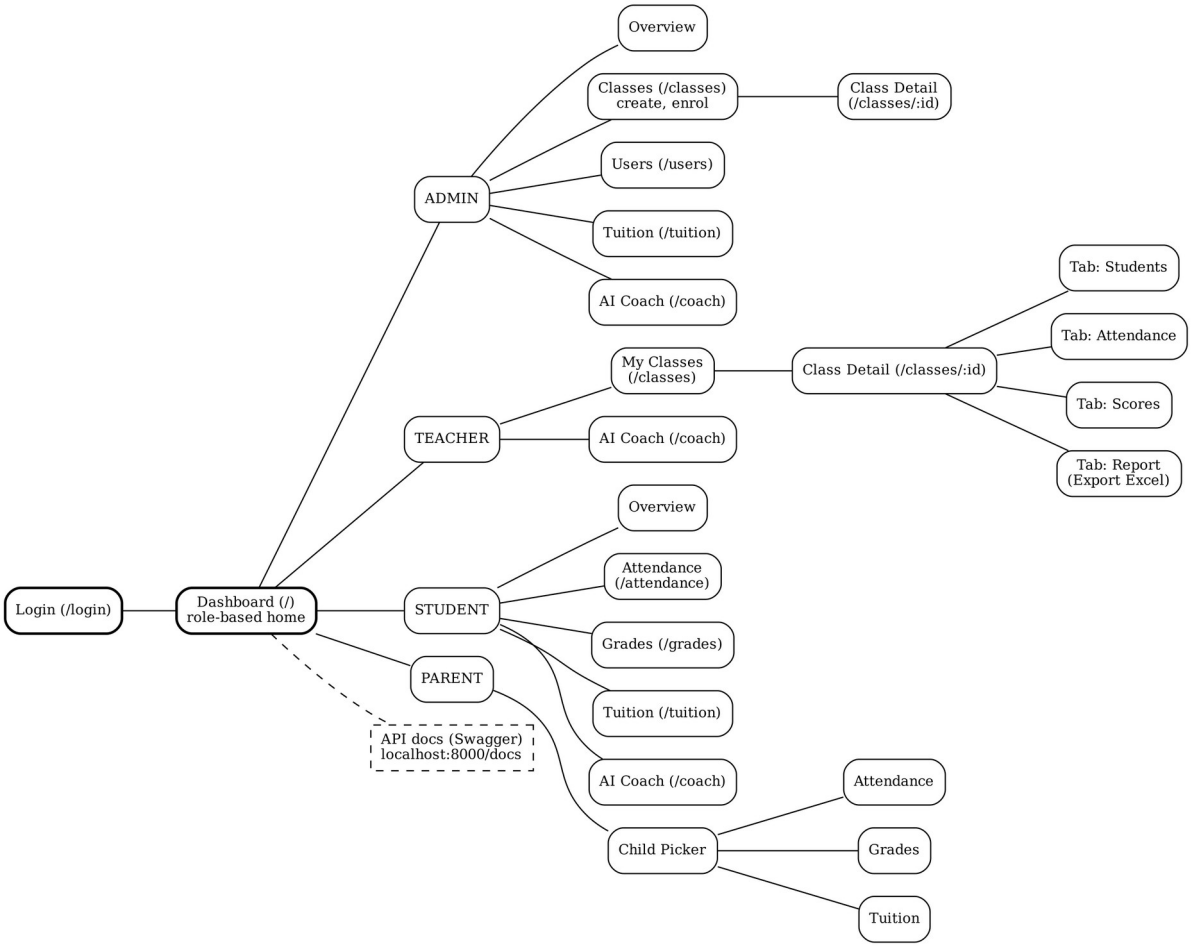
Serial	Fields	Data type	Keys	Constraints	Description
1	id	integer	PK	Auto increment, Not Null	Session identifier
2	user_id	integer	FK	References users.id, On delete cascade, Index	User who asked
3	kind	varchar(16)		Not Null, in (writing, speaking)	Type of text
4	input_text	text		Not Null	Text submitted by the user
5	feedback	text		Not Null	Feedback returned
6	provider	varchar(32)		Not Null, in (gemini, rules)	Feedback source
7	created_at	timestamptz		Not Null, Default now (UTC), Index	Creation time (daily limit)

B. Relationship Diagram



	Prepared by (Student)	Approved by (Faculty)
	Project Group No: 3	TRUONG TRUNG
Signature		
Date		

SiteMap



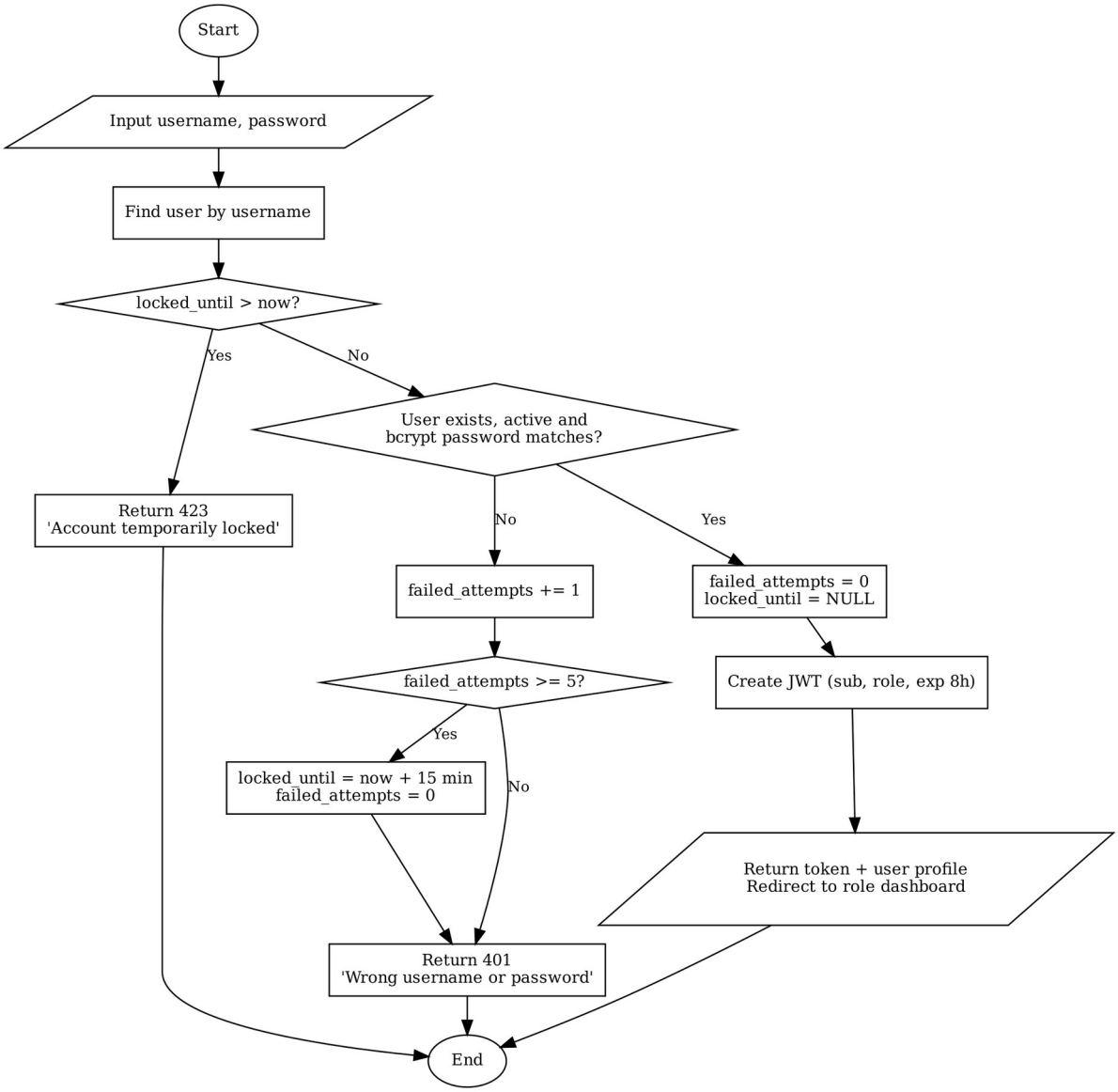
Every route except /login is protected. The web interface checks the role to show the right menu, and the API checks the role and data scope again on each request.

	Prepared by (Student)	**Approved by (Faculty)**
	Project Group No: **3**	TRUONG TRUNG
Signature		
Date		

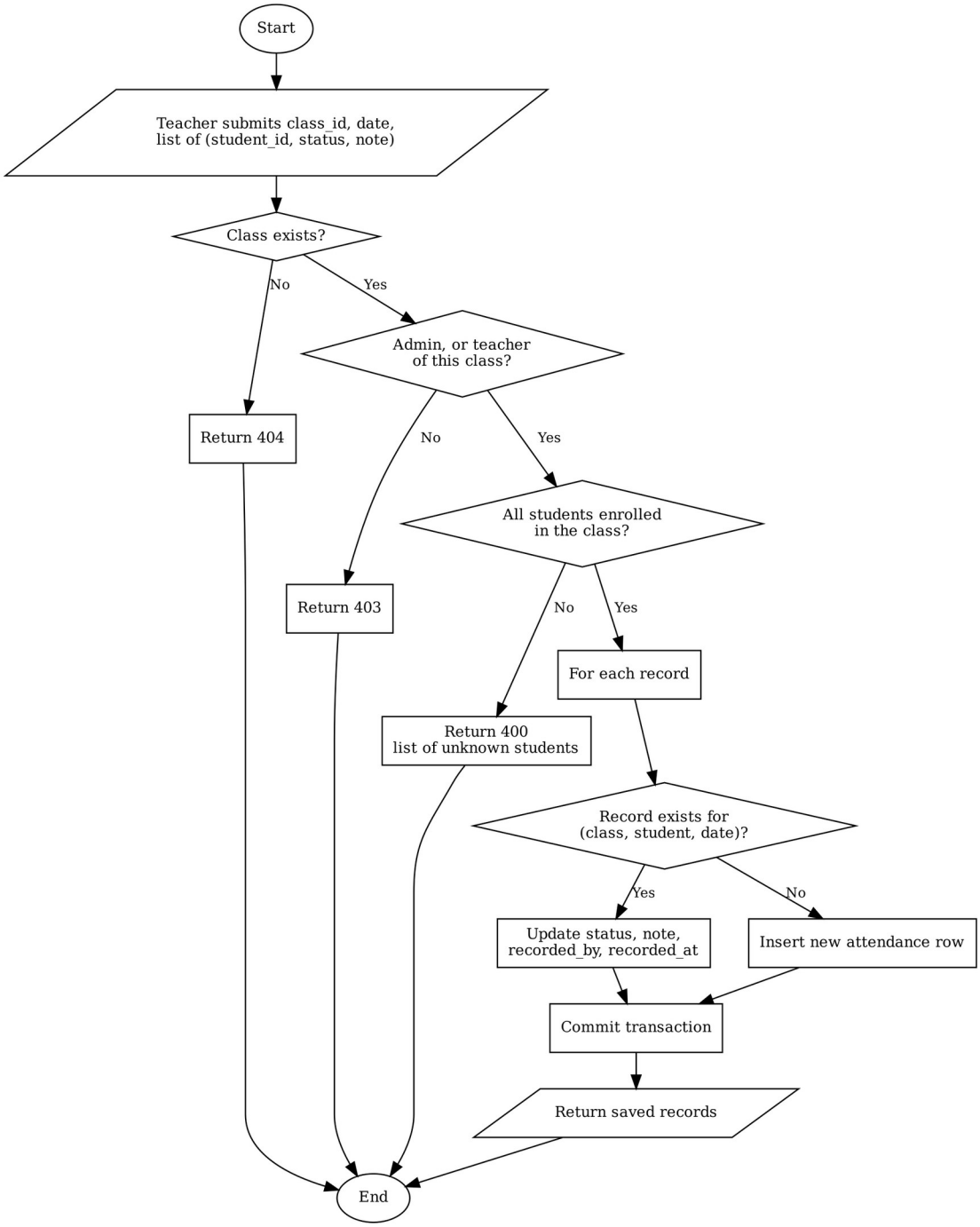
Algorithm

This section describes the algorithms of the three most important processes of the system.

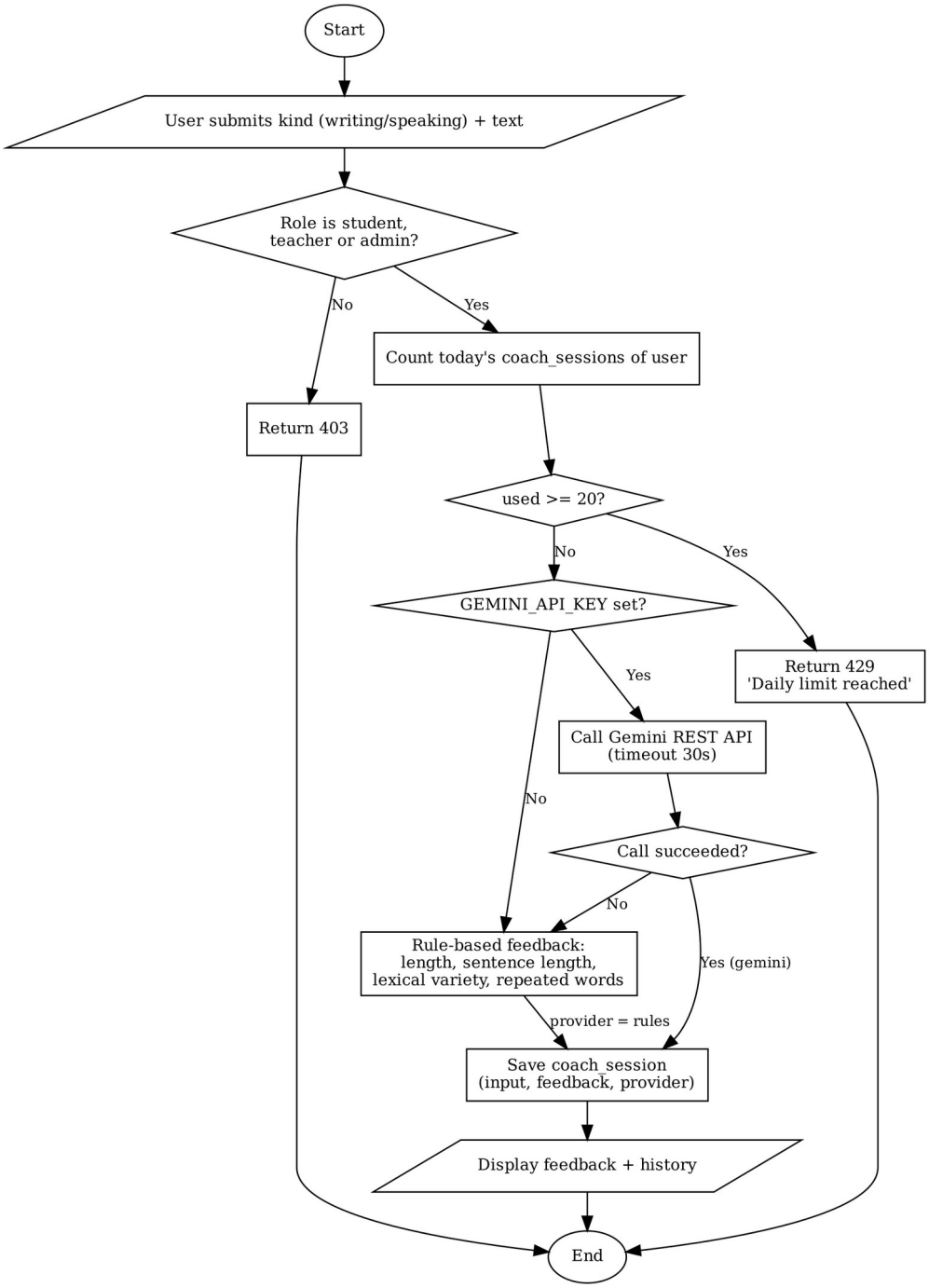
1. Login with temporary account lock



2. Teacher saves class attendance (batch upsert)



3. AI Coach feedback with daily limit and fallback

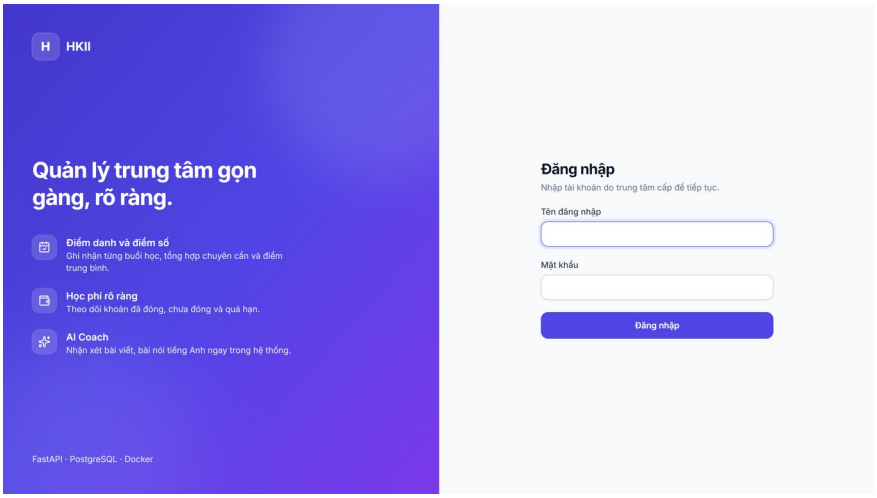


Review 03

Screen shots

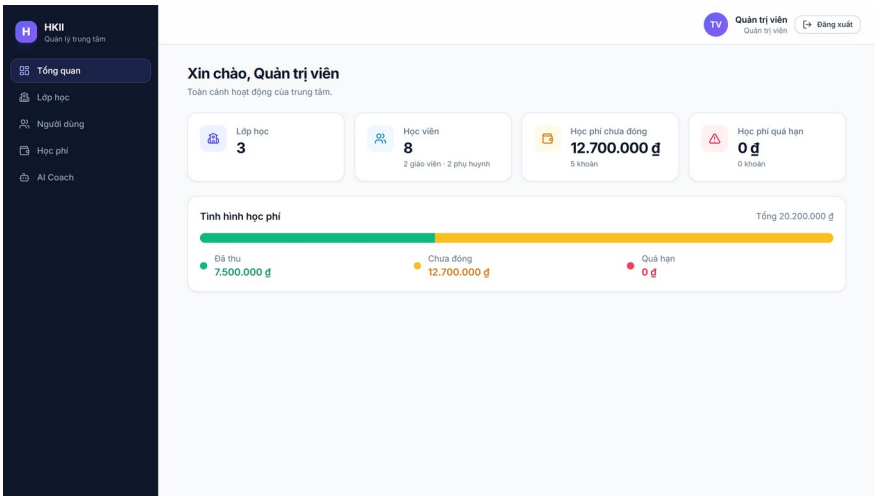
This section describes the main screens of the web interface (React, served by nginx at http://localhost:8080) and the API documentation page.

1. Login



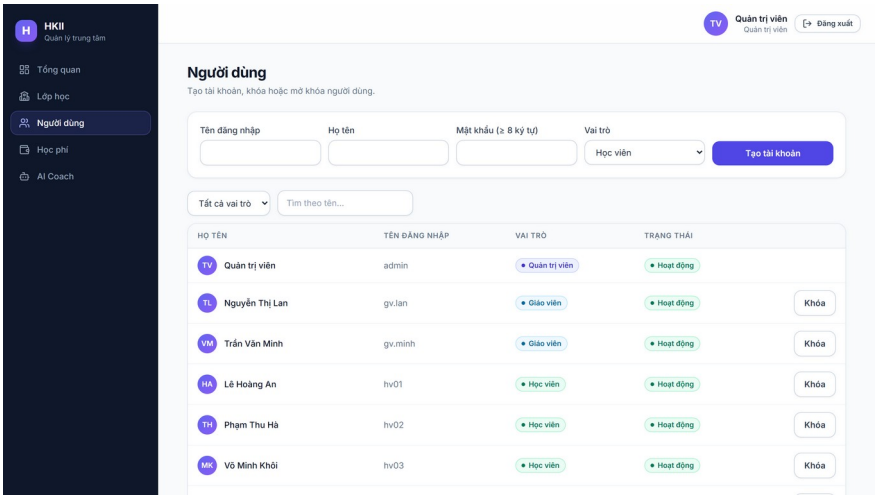
- **Description:** The entry page of the web interface. Users enter username and password; wrong credentials show one generic message, and repeated failures lock the account for 15 minutes.
- **From:** Root URL or any protected route when the user is not logged in.
- **To:** Dashboard of the user's role after a successful login.

2. Admin Dashboard



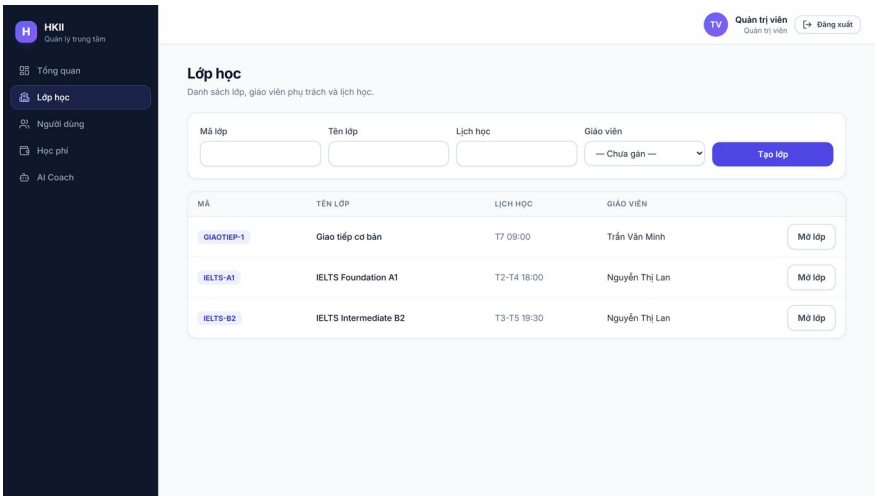
- **Description:** Overview for the administrator: number of users, classes and tuition figures by status.
- **From:** Opened automatically after an admin logs in, or from the Overview menu.
- **To:** Classes, Users, Tuition and AI Coach pages.

3. User Management



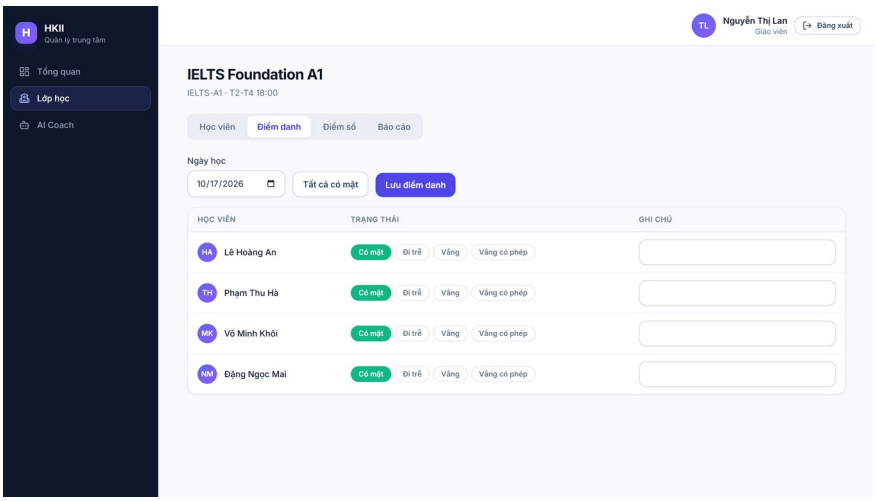
- **Description:** Admin creates accounts for all roles, edits full name, role and password, and activates or deactivates accounts.
- **From:** Users menu (admin only).
- **To:** Stays on the same page and refreshes the list after each change.

4. Class Management



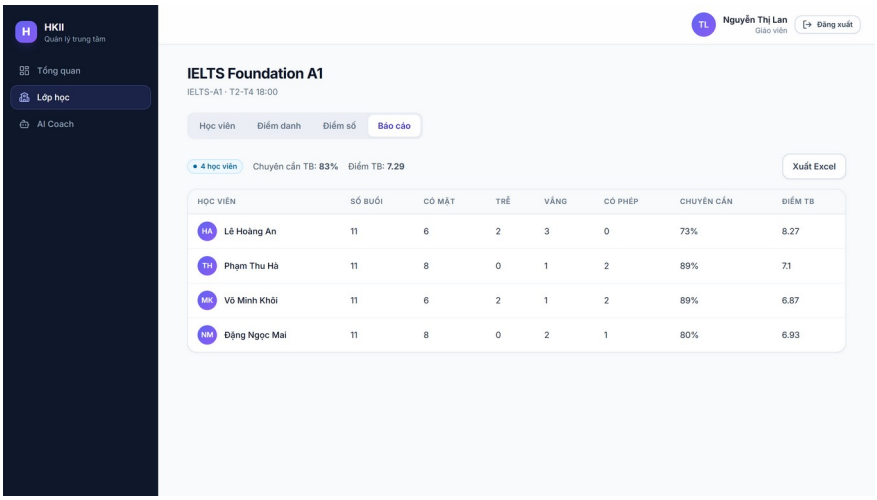
- **Description:** List of classes with code, name, schedule and teacher. Admin creates a class and enrols students; teachers see only their own classes.
- **From:** Classes menu (admin and teacher).
- **To:** Class Detail page of the selected class.

5. Class Detail: Attendance



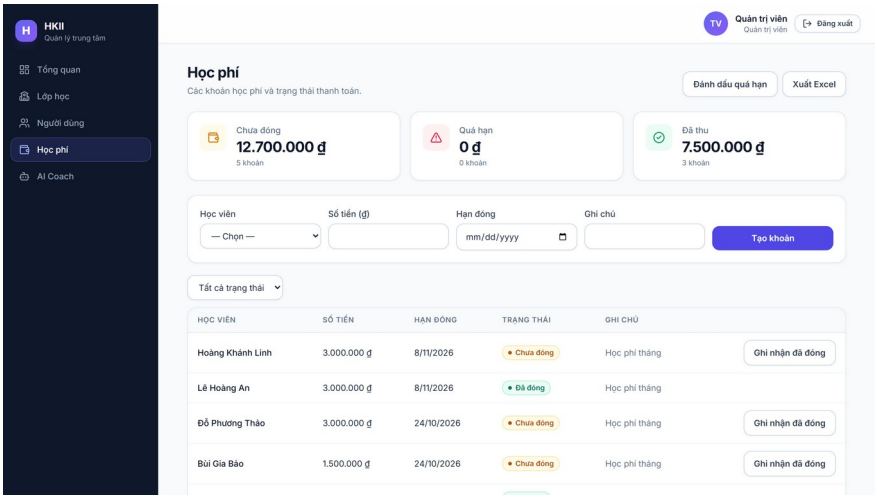
- **Description:** Teacher picks a date and marks every student as present, late, absent or excused, then saves the whole sheet in one request. Saving again for the same date updates the records.
- **From:** Class Detail page, Attendance tab.
- **To:** Students, Scores and Report tabs of the same class.

6. Class Detail: Report



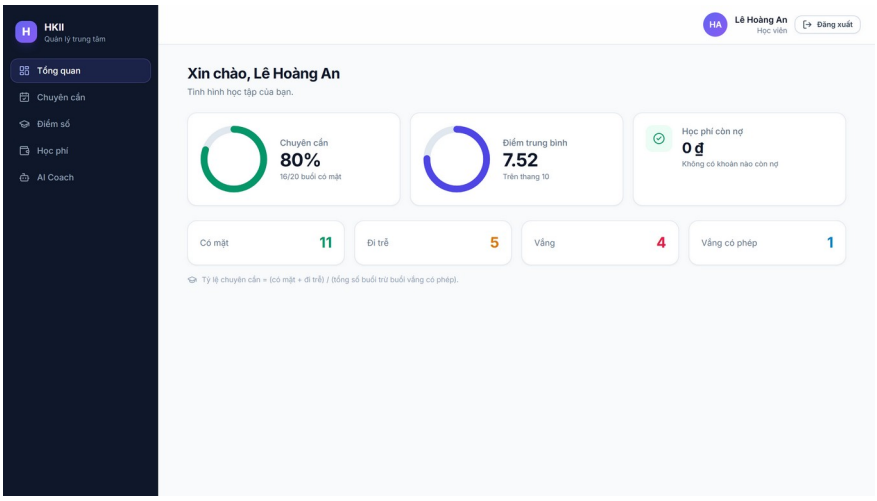
- **Description:** Class summary computed with pandas: attendance rate and average score of each student. The Export Excel button downloads an .xlsx file.
- **From:** Class Detail page, Report tab.
- **To:** Downloads the report file; other tabs of the class.

7. Tuition Management



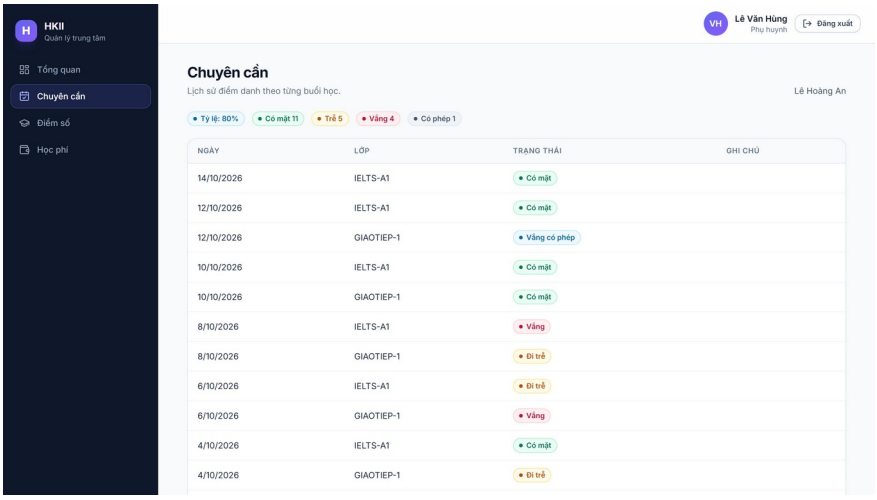
- **Description:** Admin creates tuition items, records payments and runs Mark overdue. Totals by status are shown at the top. Students and parents see a read-only version.
- **From:** Tuition menu (admin, student, parent).
- **To:** Stays on the same page; export of tuition data.

8. Student Dashboard



- **Description:** Personal overview for a student: attendance rate, latest scores and tuition status.
- **From:** Opened automatically after a student logs in.
- **To:** Attendance, Grades, Tuition and AI Coach pages.

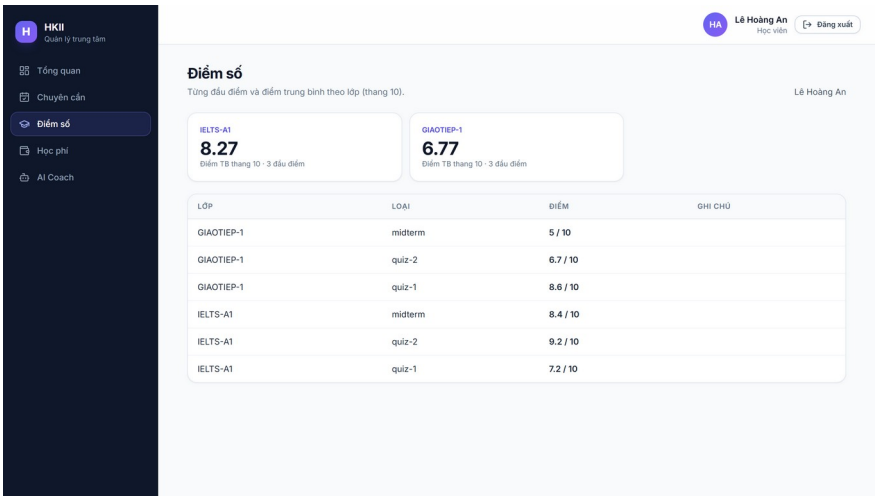
9. Parent: Child Attendance



- **Description:** A parent selects one of his or her children and views the attendance history. Data of other students cannot be opened.
- **From:** Attendance menu after login as parent; child picker at the top.
- **To:** Grades and Tuition pages of the selected child.

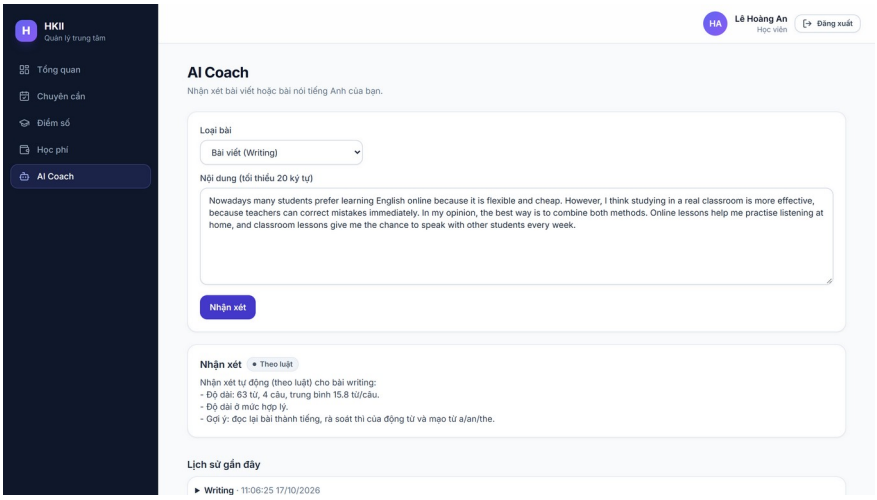
	Prepared by (Student)	**Approved by (Faculty)**
	Project Group No: **3**	TRUONG TRUNG
Signature		
Date		

10. Grades



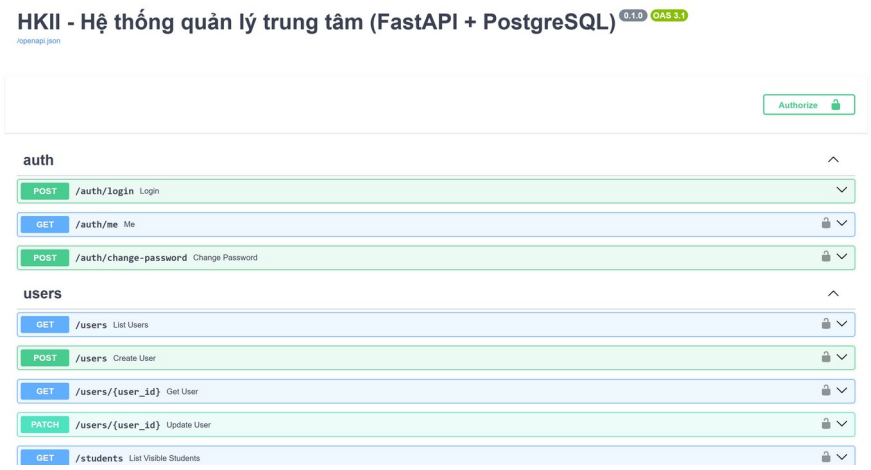
- **Description:** Scores of the student (or the selected child) grouped by class, with the average score of each class.
- **From:** Grades menu (student, parent).
- **To:** Dashboard, Attendance and Tuition pages.

11. AI Coach



- **Description:** User chooses writing or speaking, pastes a text and receives feedback (strengths, errors with corrections, suggestions). History of previous sessions is listed below. Limit: 20 requests per day.
- **From:** AI Coach menu (admin, teacher, student).
- **To:** Stays on the same page and adds the new feedback to the history.

12. API Documentation (Swagger UI)



- **Description:** Interactive documentation generated by FastAPI: 36 operations on 27 paths, grouped by module, with an Authorize button for the JWT token.
- **From:** http://localhost:8000/docs
- **To:** Each endpoint can be tried directly from the page.

Common interface properties (applied to all screens)

Properties	Value
Document theme and colour scheme	Light mode. Primary: Indigo (#4f46e5); text: Slate (#1e293b); background: Slate 50 (#f8fafc)
Form / Page background colour	White (#ffffff) cards with rounded corners on a light slate page
Title font	Inter Variable, semibold, 18 to 24 px
Title colour / alignment	Slate 800 (#1e293b), left-aligned
Background colour of controls	White (#ffffff) with slate 300 border (#cbd5e1), rounded 12 px
Foreground colour of controls	Slate 800 text, slate 400 placeholder
Control caption: size / style	14 px, medium weight, slate 600
Focus state	Indigo 500 border with indigo 100 focus ring

Properties	**Value**
Control caption and controls alignment	Labels above inputs, left-aligned
Command button alignment	Right of the form header or below the form; primary buttons in indigo

Testing document

A. Automated tests (pytest, back end). Command: docker compose run --rm --no-deps api python -m pytest -q. Result on 23/10/2026: 15 passed.

Sr.No	Features Tested	Expected Result	Result
1	Health check (GET /health)	Returns 200 and status ok	Pass
2	Login with wrong password	Returns 401 with one generic message (does not reveal whether the account exists)	Pass
3	Account lock after repeated failures	After 5 wrong passwords the account is locked; login returns 423	Pass
4	Call a protected endpoint without token	Returns 401	Pass
5	Student creates a class or a user	Returns 403	Pass
6	Parent views another student's data	Own child visible; other student returns 403	Pass
7	Teacher manages another teacher's class	Returns 403	Pass
8	Teacher views tuition	Returns 403	Pass
9	Attendance saved twice for the same date	Records updated (upsert), no duplicate row	Pass
10	Attendance for a student not in the class	Returns 400 with the list of unknown students	Pass
11	Score validation and summary	Value above max_value rejected (422); average per class correct	Pass
12	Mark overdue and pay tuition	Past-due unpaid item becomes overdue; pay sets paid and paid_at; paying twice returns 409	Pass
13	Class report and Excel export	Summary returns attendance rate and average; export returns a valid .xlsx file	Pass

Sr.No	Features Tested	Expected Result	Result
14	AI Coach without Gemini key	Feedback generated with provider = rules and saved in history	Pass
15	Parent uses the AI Coach	Returns 403	Pass

B. Manual test cases (web interface and deployment).

Sr.No	Features Tested	Expected Result	Result
16	Login form: valid credentials of each role	Redirects to the dashboard of that role with the right menu	Pass
17	Login form: empty username or password	Form shows required message; no request sent	Pass
18	Admin creates a user with a username already used	Error message, no new account	Pass
19	Admin creates a class and enrols 3 students	Class appears in the list; roster shows 3 students	Pass
20	Teacher saves the attendance sheet	Success message; attendance rate updated on the Report tab	Pass
21	Teacher exports the class report	Browser downloads an .xlsx file that opens in Excel	Pass
22	Admin records a tuition payment	Status changes to paid; totals at the top are updated	Pass
23	Parent switches between two children	Attendance, grades and tuition change to the selected child	Pass
24	Student sends a text to the AI Coach	Feedback shown and added to the history	Pass
25	Swagger UI: Authorize with a token and call /auth/me	Returns the profile of the logged-in user	Pass

Sr.No	Features Tested	Expected Result	Result
26	Clean run on another machine: docker compose up --build	Tables created by Alembic, sample data loaded, web opens at port 8080	Planned (before 20/11/2026)
27	AI Coach daily limit (21st request in a day)	Returns 429 with a clear message	Planned (test being added)

Final checklist

Sr.No	Aspects Tested	Suggestion / Remarks
1	Are all the users able to view the images and links?	Yes, all icons, images and navigation links render correctly.
2	Have all the views, modules and controllers been properly integrated and is the site function as a single page application?	Yes, the React single page application calls one FastAPI back end through /api; the five modules share one database and one login.
3	Are the GUI content devoid of spelling mistakes?	Yes, labels and messages have been proofread.
4	Is the application user-friendly?	Yes, each role sees only its own menu; tables and forms follow one consistent style.
5	Is the application launching correctly in all popular browsers?	Yes, tested on Google Chrome, Microsoft Edge and Mozilla Firefox.
6	Are all the forms validated with proper criteria?	Yes, Pydantic validates every request on the server (length, pattern, ranges) and the forms validate required fields.
7	Do all text links lead to the appropriate page?	Yes, React Router routes are protected by role; unknown routes return to the dashboard.
8	Do all image links lead to the appropriate page?	Yes.
9	Are all the images and links clearly visible on the page?	Yes, light theme with good contrast.
10	Does the web page work properly in all the tested browsers?	Yes.
11	Does the web page take too long to be loaded fully?	No, pages load in under 2 seconds on a local Docker deployment.

12	Is the navigation sequence correct through all the web pages on the site?	Yes, see the Site Map.
13	Is the JavaScript code working as expected in all click events?	Yes, buttons, tabs, child picker and Excel download work as expected.
14	Is the database securely protected against unauthorised access?	Yes, passwords hashed with bcrypt, JWT on every request, role and data scope checked in deps.py, temporary lock after 5 failed logins, secrets kept in .env (not committed).
15	Does the application handle exceptions gracefully?	Yes, clear HTTP errors (400, 401, 403, 404, 409, 423, 429) with messages; the AI Coach falls back to rule-based feedback when Gemini fails.

Task Sheet

No	Member	Contents	Table Name	Planned Start	Actual Start	Days	Status
1	Thái Phát	Requirement analysis: problem definition, 4 roles, 5 modules, acceptance criteria		09/10/2026	09/10/2026	3	Done
2	Thái Phát	Database design: ERD, 8 tables, keys and constraints	all tables	11/10/2026	11/10/2026	3	Done
3	La Vĩ Quyền	Back end: FastAPI, models, Alembic migration, JWT, role checks, 36 API operations, 15 tests	all tables	14/10/2026	14/10/2026	1	Done
4	La Vĩ Quyền	Front end React + nginx; Docker Compose (db, api, web)		14/10/2026	14/10/2026	1	Done
5	La Vĩ Quyền	Run guide, demo script, presentation; new interface design		17/10/2026	17/10/2026	1	Done
6	Both	Task assignment, 15 GitHub Issues for remaining work		19/10/2026	19/10/2026	1	Done
7	Thái Phát	Documentation Review 1 and Review 2, 1st Status Report		20/10/2026	20/10/2026	4	Done

	Prepared by (Student)	**Approved by (Faculty)**
	Project Group No: **3**	TRUONG TRUNG
Signature		
Date		

No	Member	Contents	Table Name	Planned Start	Actual Start	Days	Status
8	Thái Phát	Screens: change password, parent-child link, edit score; tests for enrolment and user management	users, parent_students, scores, enrollments	26/10/2026		7	Planned
9	La Vĩ Quyền	AI Coach daily-limit test; GitHub Actions CI (pytest + npm build); tests on PostgreSQL; security review of .env	coach_sessions	26/10/2026		7	Planned
10	Thái Phát	Review 3: screenshots, testing document, final checklist		02/11/2026		10	Planned
11	La Vĩ Quyền	Installation guide, user guide, clean-machine run		09/11/2026		7	Planned
12	Both	Cross testing (each member tests the other's modules), fix bugs		16/11/2026		4	Planned
13	Both	Submit design documents (20/11), final project (02/12), slides, demo video, defence rehearsal (05/12)		20/11/2026		15	Planned

	Prepared by (Student)	Approved by (Faculty)
	Project Group No: **3**	TRUONG TRUNG
Signature		
Date		